Syntax and how to write a terraform files:

[Master these and you can confidently say you know terraform]

1. The Basic Anatomy of a Block

Everything in Terraform is organized into **blocks**. A block is a container for other content and usually represents the configuration of some kind of object, like a virtual machine or a network.

Here is the general structure:

```
block_type "label_1" "label_2" {

  argument_name = "argument_value"
}
```

- **block_type**: Tell Terraform what kind of object you are configuring (e.g., resource, provider, variable, output).
- **labels**: Names or identifiers for the block. Some blocks have zero labels, some have one, and some (like resources) require two.
- **{ in curly braces }**: The body of the block.
- **arguments**: Inside the body, you define key-value pairs that set the specific details for that block.

---

2. The Core Block Types

To build infrastructure, you will typically use these four main types of blocks:

A. The terraform Block

This block configures Terraform itself, such as specifying which cloud providers you need and which version of Terraform is required.

```
terraform {

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

B. The `provider` Block

This tells Terraform *where* to build your infrastructure (AWS, Azure, Google Cloud, etc.) and gives it the necessary configuration (like the region).

```
provider "aws" {

  region = "us-east-1"
}
```

C. The `resource` Block (The most important one!)

This defines the actual piece of infrastructure you want to create, like a server, a database, or a virtual network. It always takes **two labels**: the *resource type* (defined by the provider) and the *resource name* (defined by you, used to reference it elsewhere in your code).

```
# block_type | resource_type | resource_name

resource      "aws_instance"  "my_web_server" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "MyWebServer"
  }
}
```

D. `variable` and `output` Blocks

- **Variables** let you pass values into your configuration so you don't have to hardcode everything.
- **Outputs** print out specific information to your terminal after Terraform builds your infrastructure (like an IP address).

```
Terraform


variable "server_port" {
  description = "The port the server will use for HTTP requests"
  type        = number
  default     = 8080
}

output "server_public_ip" {
  description = "The public IP address of the web server"
  value       = aws_instance.my_web_server.public_ip
}
```

---

3. Syntax Rules to Remember

- **Comments:** * Use `#` for single-line comments (this is the most common standard).
    - Use `//` for single-line comments (also accepted).
    - Use `/* ... */` for multi-line comments.
- **Strings, Numbers, and Booleans:**
    - Strings are always enclosed in double quotes: `"this is a string"`.
    - Numbers and booleans (`true` / `false`) do not need quotes.
- **Lists and Maps:**
    - **Lists** use square brackets: `["item1", "item2"]`
    - **Maps** (key-value pairs) use curly braces, often used for tags: `{ Name = "Web", Environment = "Dev" }`
- **Referencing other resources:** You can grab data from one resource and feed it into another using the syntax `resource_type.resource_name.attribute`. (e.g., `aws_instance.my_web_server.id`).

Example with an explanation:

Problem: Create 5 ec2 instance at the same time using terraform:

Code:

```
terraform {

  required_providers {

    aws = {

      source  = "hashicorp/aws"

      version = "~> 5.0"

    }

  }

}

provider "aws" {

  region = "ap-south-1"

}

data "aws_ami" "amazon_linux" {

  most_recent = true

  owners      = ["amazon"]



  filter {

    name   = "name"

    values = ["amzn2-ami-hvm-*-x86_64-gp2"]

  }

}
```

```
resource "aws_instance" "free_tier_ec2" {

  count         = 5

  ami           = data.aws_ami.amazon_linux.id

  instance_type = "t2.micro"



  tags = {

    Name = "free-tier-ec2-${count.index + 1}"

  }

}
```

Explanation:
Here is the block-by-block breakdown of exactly how it works:

1. The `terraform` Block (The Prerequisites)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

- **What it does:** This tells Terraform, "Before doing anything else, download the plugin (provider) that knows how to talk to AWS."
- **Syntax details:** It specifically asks for the official AWS provider maintained by HashiCorp (`source`). The `version = "~> 5.0"` part is a constraint meaning "use version 5.0, or any minor update after it (like 5.1 or 5.2), but do not jump to version 6.0."

2. The `provider` Block (The Target)

```
provider "aws" {
  region = "ap-south-1"
}
```

- **What it does:** This configures the AWS plugin you just downloaded. It tells Terraform that any AWS resources you create in this file should be built in the `ap-south-1` region (which is Mumbai, India).

3. The `data` Block (Fetching Information)

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}
```

- **What it does:** This introduces a new block type: **`data`**. While a `resource` block *creates* something, a `data` block *reads* existing information from your cloud provider so you can use it in your code.
- **Syntax details:** Here, you are asking AWS to find an AMI (Amazon Machine Image—the operating system blueprint). You are telling it to filter for images owned by "amazon", matching a specific Amazon Linux 2 naming pattern, and to grab the `most_recent` one. You named this data block `amazon_linux`.

4. The `resource` Block (Creating the Infrastructure)

```
resource "aws_instance" "free_tier_ec2" {
  count         = 5
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"

  tags = {
    Name = "free-tier-ec2-${count.index + 1}"
  }
}
```

- **What it does:** This creates the actual servers (EC2 instances).
- **Syntax details & New Concepts:**

- **`count = 5`**: This is a Terraform "meta-argument." Instead of writing this block out five separate times, you are telling Terraform to loop through this block and create 5 identical servers.
- **`ami = data.aws_ami.amazon_linux.id`**: Notice how you aren't hardcoding the AMI ID (like `ami-12345`). Instead, you are referencing the `data` block from above. You are saying, "Take the ID of the image you just looked up, and use it here."
- **`instance_type = "t2.micro"`**: This specifies the server size (which happens to be eligible for the AWS free tier).
- **`${count.index + 1}`**: This is called **string interpolation**. Because you used `count = 5`, Terraform keeps track of which loop it is on, starting at `0` (`count.index`). By adding `+ 1`, your servers will be automatically tagged with names like `free-tier-ec2-1`, `free-tier-ec2-2`, up to `free-tier-ec2-5`.